

UNIVERSIDAD TECNOLÓGICA DE PANAMA CENTRO REGIONAL DE CHIRIQUI
LABORATORIO DE DESARROLLO DE MÓVILES

**1. ¿Qué ventajas ofrece el diseño adaptativo en comparación con el diseño fluido?
En qué escenarios es preferible cada enfoque ?**

R: El fluido es el que estira y encoge el contenido. El adaptativo cambia el layout completo dependiendo del dispositivo. Uso fluido para listas simples, adaptativo cuando tablet y móvil necesitan layouts completamente distintos.

2. ¿Cómo funciona el widget Wrap y en qué situaciones sería útil?

R: Es como un Row pero cuando se queda sin espacio, baja los elementos a la siguiente línea. Funciona para chips, etiquetas o botones de filtros.

3. ¿Cuál es el propósito de un Theme en flutter y cómo se aplica a una aplicación?

R: Es el "vestido" global de la app: colores, fuentes, formas. Lo defines una vez y todos los widgets lo heredan automáticamente.

4. Explica cómo se utiliza la clase TextTheme para definir y aplicar estilos de texto en una aplicación flutter?

R: En vez de poner `fontSize: 24` en cada Text, se define con roles como el `displayLarge` o `bodyMedium` en el tema y se reutiliza en toda la app con `Theme.of(context).textTheme.bodyMedium`.

5. Describe algunas técnicas avanzadas de estilo que se pueden utilizar para mejorar la apariencia de una aplicación flutter.

R: **Técnicas avanzadas de estilo** `ShaderMask` para gradientes, `ClipRRect` para bordes redondeados, `BoxDecoration` combinando sombras y gradientes, `AnimatedContainer` para transiciones suaves y `ThemeExtension` para propiedades personalizadas en el tema.

6. ¿Cómo puedes detectar la plataforma en la que se ejecuta tu aplicación Flutter ?

R: **Detectar plataforma** Con `Platform.isAndroid`, `Platform.isIOS`, o `kIsWeb` para web.

7. ¿Cómo puedes personalizar los estilos de tu aplicación Flutter ?

R: Tres niveles: global con `ThemeData`, local con `Theme(...copyWith(...))` y por widget con el parámetro `style`: directo.

8. ¿Cuándo debes usar setState() en Flutter ?

R: Cuando el estado es solo de ese widget y es algo simple: un contador, un toggle, un formulario local. Si el estado lo necesitan otros widgets, mejor usar `Provider` o `Riverpod`.

9. En qué situaciones serían más apropiados usar ListView en lugar de GridView.

R: ListView cuando los items son de altura variable o tienen texto largo, como chats o noticias. GridView cuando los items son visuales y del mismo tamaño, como una galería o catálogo de productos.

10. Investiga y compara brevemente Provider, Riverpod y Bloc.

R: Provider es el más simple, bueno para apps pequeñas. Riverpod es una versión mejorada de Provider, más segura y escalable. Bloc es el más estructurado y verboso, ideal para equipos grandes con lógica compleja.

11. Cuáles son las mejores prácticas para manejar errores al consumir APIs

R: Siempre usar try/catch separando errores de red, HTTP y parseo. Mostrarle al usuario mensajes amigables, nunca el error técnico crudo. Validar el statusCode antes de procesar la respuesta.

12. Cómo se puede mapear datos JSON a clases de modelo en Flutter ?

R: Crear una clase con un constructor fromJson() que lee el Map y un toJson() que lo convierte de vuelta. Para proyectos grandes, el paquete json_serializable lo genera automáticamente.

13. Qué es FutureBuilder y cómo se usa para mostrar datos asíncronos ?

R: Es un widget que maneja solo los tres estados de una llamada asíncrona: cargando, error y datos listos. Evita hacer setState() manual para mostrar un spinner mientras carga.

14. Cuáles son las limitaciones de setState en aplicaciones flutter complejas?

R: No sirve para compartir estado entre pantallas distintas, reconstruye más widgets de los necesarios, mezcla la lógica con la UI y es difícil de testear.

15. Cómo funciona la arquitectura Provider ?

R: Se coloca un ChangeNotifierProvider arriba en el árbol, el modelo llama al notifyListeners() cuando algo cambia, y los widgets con Consumer o context.watch se reconstruyen solos. Separa la lógica de la UI de forma limpia.